

GPT FROM SCRATCH

Professional Project Plan & Execution Pipeline

Implementing a GPT-Style Transformer from Scratch in PyTorch

Framework	PyTorch 2.x
Hardware	RTX 3050 12GB VRAM / 16GB RAM
Dataset	WikiText-2 (primary)
Timeline	25 Days
Advanced Feature	RoPE (Rotary Positional Embeddings)

1. Project Overview & Revised Scope

This document defines the revised, production-grade plan for building a GPT-style decoder transformer from scratch. Every decision below has been made to maximize quality and deliverability within a 25-day window on consumer hardware. Shortcut-prone areas have been explicitly addressed.

1.1 Core Objectives

- Implement a fully configurable GPT-style transformer in pure PyTorch 2.x
- Train on WikiText-2 with a professional training loop (mixed precision, scheduler, gradient accumulation)
- Measure and report perplexity across two model sizes (scaling study)
- Implement and compare greedy, top-k, top-p, and temperature sampling
- Integrate RoPE (Rotary Positional Embeddings) as the advanced architectural feature
- Optionally compare against a 2-layer LSTM baseline if time permits
- Produce a clean GitHub repository and a structured report

1.2 What Was Deliberately Removed or Replaced

The following items from earlier drafts were cut or replaced based on time-cost vs. value analysis:

Original Plan	Decision	Reason
Train SentencePiece from scratch	Use HuggingFace tokenizers (BPE)	Saves 3–5 days, same conceptual value
85-day pipeline	Compressed to 25 days	Realistic with AI assistance
Multiple advanced features	RoPE only	Depth > breadth in 25 days
Manual logging (matplotlib)	Weights & Biases (wandb)	Professional training curves for report
No mixed precision	torch.cuda.amp throughout	~50% speedup, fits larger batches

2. Technical Decisions & Justifications

2.1 Dataset: WikiText-2

WikiText-2 is the correct choice for this project under time and compute constraints. It is large enough (~2M training tokens) to demonstrate real language modeling behavior, well-benchmarked in the literature (so your perplexity numbers are directly comparable), and fast to tokenize and load. The move to WikiText-103 or OpenWebText is a natural next step and should be discussed as future work in your report — this framing is honest and professional.

2.2 Tokenizer: HuggingFace BPE (Not From Scratch)

Training SentencePiece from scratch has educational value but costs 3–5 days with minimal payoff given your timeline. The correct approach is to use HuggingFace's tokenizers library with BPE and allocate the saved time to model quality and evaluation depth. In your report, dedicate half a page to explaining how BPE works (merge operations, vocabulary construction, the encode/decode cycle) — this demonstrates the theory without requiring you to reimplement it.

2.3 Training Loop: Non-Negotiable Components

Critical

The following four components are not optional. Missing any one of them will produce a noticeably worse model and a weaker project.

Mixed Precision (`torch.cuda.amp`): Wrap your forward pass and loss computation in `autocast()`. Use `GradScaler` for the backward pass. This alone gives you ~50% speed improvement and allows larger batch sizes on your 3050.

Gradient Accumulation: Set an `accumulation_steps` parameter (recommended: 4–8). This lets you simulate an effective batch size of 128–256 even if your GPU can only fit 32 samples at once. Critical for stable training.

LR Scheduler (Warmup + Cosine Decay): Linear warmup for the first ~500 steps, then cosine annealing to near-zero. Without this, AdamW on transformers is significantly less stable. Define `warmup_steps` in your config.

`torch.compile()`: Call `torch.compile(model)` after model instantiation (PyTorch 2.x only). One line, free 10–30% speedup from kernel fusion. No code changes needed elsewhere.

2.4 Advanced Feature: RoPE

Rotary Positional Embeddings replace the learned positional embedding table in the standard GPT architecture. Instead of adding a fixed or learned vector to the token embedding, RoPE rotates the query and key vectors in attention by an angle proportional to their position. This is the approach used in LLaMA, Mistral, and most modern production LLMs.

From an implementation standpoint, the change is localized to your `attention.py` file — you modify how Q and K are computed before the dot product. The rest of the architecture is unchanged. This makes it a contained, testable modification that is directly relevant to the current state of the field.

2.5 Architecture Configuration

Hyperparameter	Small Model (~30M)	Medium Model (~60M)
Layers (n_layer)	6	8
Attention Heads (n_head)	8	8
Embedding Dim (d_model)	512	768
FFN Hidden Dim (4× d_model)	2048	3072
Context Length (block_size)	256	256
Batch Size (effective)	128 (accum × 4)	128 (accum × 8)
Learning Rate	3e-4	2e-4
Weight Decay	0.1	0.1
Dropout	0.1	0.1

3. Final Project Structure

This structure is finalized and should be created on Day 1 before any code is written. Deviating from it mid-project creates confusion and messy commits.

Path	Purpose
gpt-from-scratch/	Project root
data/raw/	Raw downloaded text files (train.txt, val.txt)
data/processed/	Cleaned text + encoded token ID arrays (.npy)
tokenizer/	BPE tokenizer config and vocab files
model/attention.py	Multi-head attention + causal mask + RoPE implementation
model/transformer_block.py	LayerNorm, attention, MLP, residual connections
model/gpt.py	Full model: embeddings, stacked blocks, linear head
training/config.py	All hyperparameters as a single dataclass
training/dataset.py	PyTorch Dataset: loads .npy, returns shifted (x, y) pairs
training/train.py	Full training loop: AMP, scheduler, grad accumulation, wandb
evaluation/generate.py	Text generation: greedy, top-k, top-p, temperature
evaluation/metrics.py	Perplexity calculation over validation set
utils/helpers.py	Shared utilities: seed setting, checkpoint save/load, logging
experiments/logs/	wandb run artifacts + any local CSVs
experiments/checkpoints/	Saved .pt model files with step/epoch metadata
tests/	Unit tests for attention mask and dataset shifting (write on Day 8)
README.md	Architecture diagram, training instructions, results table
requirements.txt	Pinned package versions

4. Phase-by-Phase Plan

Phase 1 — Environment & Repository Setup (Day 1)

The goal on Day 1 is a clean, committed, runnable project skeleton. No model code yet.

- Create the full folder structure defined in Section 3 with placeholder files
- Initialize Git repository and make the first commit with the empty structure
- Create and activate a Python virtual environment (venv or conda)
- Install: torch torchvision torchaudio (CUDA 12.x build), numpy, tqdm, wandb, datasets, tokenizers
- Pin all versions in requirements.txt immediately
- Verify CUDA availability: `python -c "import torch; print(torch.cuda.is_available())"`
- Set up wandb account and run wandb login
- Write config.py as a Python dataclass with all hyperparameters defined (even if unused yet)

Day 1 Exit Criterion

git log shows a clean initial commit. `torch.cuda.is_available()` returns True. `config.py` exists and imports cleanly.

Phase 2 — Data Pipeline (Days 2–3)

Step 2.1 — Download Dataset: Use HuggingFace datasets library to pull WikiText-2. Save train and validation splits as plain text files to `data/raw/`. One script, ~10 lines.

Step 2.2 — Clean Text: Strip empty lines, normalize whitespace, remove wiki section markers (`=` = Section = `=`). Save to `data/processed/`. Keep the cleaning logic simple and reproducible.

Step 2.3 — Tokenize: Use HuggingFace tokenizers to train a BPE tokenizer on the training text with vocabulary size 8192 (your primary run) and 16384 (for the vocabulary size comparison). Save tokenizer files to `tokenizer/`. Encode both splits to numpy arrays and save as `train_ids.npy` and `val_ids.npy`.

Step 2.4 — Verify: Print token count for train and val sets. Decode a 50-token sample and verify it reads like normal English. This takes 5 minutes and catches broken tokenization before it wastes training time.

Day 3 Exit Criterion

`data/processed/train_ids.npy` and `val_ids.npy` exist. A sample decode produces readable text. Token counts are printed and logged.

Phase 3 — Model Implementation (Days 4–9)

This is the most technically demanding phase. Budget the most time here and do not rush it. A bug introduced here will surface as a silent failure during training — the loss will decrease but the model will not learn language.

3.1 — Causal Attention (`attention.py`) — Days 4–5

Implement in this order, testing each component before moving to the next:

1. Linear projections for Q, K, V (no bias)
2. Split into multiple heads and reshape to (batch, heads, seq_len, head_dim)
3. Scaled dot-product: $(Q @ K.T) / \sqrt{\text{head_dim}}$

4. Causal mask: register_buffer a lower-triangular boolean mask. Apply it with masked_fill before softmax. Test this mask explicitly — print it for a sequence length of 6 and verify by eye.
5. Softmax + dropout
6. Merge heads and output projection
7. Add RoPE rotation to Q and K (after projection, before dot-product). Implement the rotation matrix using sin/cos precomputation.

Masking Test (Mandatory)

After implementing the mask, write a standalone test: initialize random Q, K, V tensors, run attention, and verify that position i in the output depends only on positions $0..i$, never on $i+1$ onward. Print a small attention weight matrix and verify it is lower-triangular after softmax.

3.2 — Transformer Block (transformer_block.py) — Day 6

Assemble the block in this order: Pre-norm LayerNorm → Attention → residual add → Pre-norm LayerNorm → MLP (GELU activation, $4\times$ expansion) → residual add. Use pre-norm (LayerNorm before the sub-layer), not post-norm. Pre-norm is more stable and is what GPT-2 and its descendants use.

3.3 — Full GPT Model (gpt.py) — Days 7–8

Wire together: Token Embedding ($\text{nn.Embedding, vocab_size} \times \text{d_model}$) → Dropout → N stacked TransformerBlocks → Final LayerNorm → Linear head ($\text{d_model} \times \text{vocab_size}$). The positional embedding table is omitted since RoPE handles position. Weight-tie the token embedding and the output linear head (they share the same weight matrix — this is standard in GPT-class models and reduces parameters).

3.4 — Dataset Class (dataset.py) — Day 9

Load train_ids.npy into memory. For each index i , return $x = \text{tokens}[i : i+\text{block_size}]$ and $y = \text{tokens}[i+1 : i+\text{block_size}+1]$. The shift-by-one is the entire logic of next-token prediction. Verify this by printing the first (x, y) pair and confirming that $y[0] == x[1]$, $y[1] == x[2]$, and so on.

Batch Shift Test (Mandatory)

Print $(x[0:5], y[0:5])$ from your DataLoader. Confirm that y is x shifted by exactly one position. This is the second most common silent bug in GPT implementations.

Phase 3 Exit Criterion

$\text{model}(x)$ produces logits of shape $(\text{batch, seq_len, vocab_size})$ without errors. Attention weights are verified lower-triangular. Dataset shifting is verified by inspection.

Phase 4 — Training Loop (Days 10–13)

4.1 — Training Loop Architecture (train.py)

Implement the loop with the following components in the exact order listed:

8. Load config, initialize model, move to CUDA
9. Call `torch.compile(model)` — one line after model init
10. Initialize AdamW with weight decay 0.1, betas (0.9, 0.95)
11. Initialize GradScaler for AMP

12. Initialize cosine LR scheduler with linear warmup (use `get_cosine_schedule_with_warmup` from HuggingFace transformers or implement manually)
13. Initialize wandb run with config dict
14. Training loop: for each micro-batch, use `autocast()`, compute loss, scale loss by `1/accumulation_steps`, call `scaler.scale(loss).backward()`. After `accumulation_steps` micro-batches: `unscale gradients`, `clip to max_norm=1.0`, `scaler.step(optimizer)`, `scaler.update()`, `scheduler.step()`, `zero_grad()`
15. Every `eval_interval` steps: compute validation loss and perplexity, log to wandb, save checkpoint if val loss is best seen

4.2 — Sanity Check Before Full Training

Before running a full training job, run a 50-step overfit test on a single batch of 4 sequences. The loss must drop from $\sim \log(\text{vocab_size}) \approx 9.0$ to near zero within 50 steps. If it does not, there is a bug in the loss computation or the batch construction. Do not proceed to full training until this test passes.

Phase 4 Exit Criterion

50-step overfit test passes. Full training run starts, loss decreases smoothly from step 1, wandb dashboard shows live curves. First checkpoint saved.

Phase 5 — Scaling Study (Days 14–18)

This phase has a hard dependency on Phase 4 completing cleanly. The training runs themselves are largely unattended — start them and let them run overnight or while doing other work.

- Run A: Train small model (~30M params) to completion on WikiText-2. Log to wandb project gpt-small.
- Run B: Train medium model (~60M params) with identical settings except model size. Log to wandb project gpt-medium.
- After both runs complete: extract final validation perplexity, training wall time, and parameter count. Format as a comparison table (this goes directly into your report).
- If time allows: run a third micro-experiment with vocab size 16384 vs 8192 on the small model. This is a 2–3 hour experiment that adds a tokenization section to your results.

Phase 5 Exit Criterion

Two complete training runs with logged curves. A comparison table with perplexity, parameter count, and training time for both models.

Phase 6 — Evaluation & Generation (Days 19–21)

6.1 — Perplexity (`metrics.py`)

Load best checkpoint for each model. Run evaluation loop over the full validation set with no gradient computation. Compute mean cross-entropy loss. Perplexity = $\exp(\text{loss})$. Report to 2 decimal places.

6.2 — Text Generation (`generate.py`)

Implement all four strategies in a single file with a shared `generate()` interface that accepts a strategy parameter:

- Greedy: $\text{argmax}(\text{logits})$ at each step. Produces deterministic but often repetitive output.
- Top-k: sample from the top-k highest probability tokens. Typical k values: 40, 50.
- Top-p (Nucleus): sample from the smallest set of tokens whose cumulative probability exceeds p. Typical p: 0.9, 0.95. Generally produces the most coherent outputs.
- Temperature: divide logits by T before softmax. $T < 1.0$ sharpens, $T > 1.0$ flattens. Combine with top-p for best results.

Generate 5 samples per strategy from the same prompt and include them verbatim in your report appendix. The contrast between greedy repetition and nucleus sampling coherence is a compelling visual result.

Phase 6 Exit Criterion

Perplexity numbers computed for both models. 20+ generation samples saved to text files. Qualitative comparison written.

Phase 7 — LSTM Baseline (Days 22–23, Optional)

Note

Only begin this phase if Phases 1–6 are complete and clean. If you are behind, skip this and use the time for code cleanup and the report. The project is fully defensible without it.

Implement a 2-layer LSTM with hidden size 512 and the same token embeddings. Train for the same number of steps as the small transformer. Compare validation perplexity. The expected result is that the transformer achieves significantly lower perplexity — this demonstrates the advantage of attention over recurrence in your report's discussion section.

Phase 8 — Cleanup & Report (Days 23–25)

8.1 — Code Cleanup

- Remove all debug print statements and commented-out code
- Add docstrings to all classes and non-trivial functions
- Verify that python train.py runs cleanly from a fresh clone of the repo
- Add a tests/ directory with the two unit tests from Phase 3 (mask test + dataset shift test)
- Write the README: architecture overview, installation instructions, training instructions, results table, and sample generations

8.2 — Report Structure

16. Introduction: motivation, what you built, why it matters
17. Architecture: describe each component (attention math, RoPE, MLP, LayerNorm, residual connections)
18. Methodology: dataset, tokenization, training procedure, hardware
19. Experiments: scaling study table, perplexity results, vocabulary size experiment if done
20. Sampling Analysis: qualitative comparison of generation strategies with example outputs
21. Discussion: what worked, what you would do differently, limitations, future work (WikiText-103, larger models)

5. 25-Day Execution Pipeline

This is the day-by-day execution schedule. Days marked with a lock icon are hard dependencies — do not start them until the prior day's exit criterion is met.

Day	Phase	Tasks	Exit Criterion
1	Setup	Folder structure, Git init, venv, install deps, verify CUDA, write config.py, wandb login	CUDA=True, config imports
2	Data	Download WikiText-2, save raw text, write cleaning script	data/raw/ populated
3	Tokenizer	Train BPE (8192 vocab), encode train+val, save .npz files, verify decode	Decode test passes
4	Model	Q/K/V projections, multi-head split/merge, scaled dot-product	Attention output shape correct
5	Model	Causal mask implementation + mandatory mask unit test + RoPE rotation on Q and K	Mask test passes ✓
6	Model	TransformerBlock: pre-norm, attention, MLP (GELU, 4× expansion), both residuals	Block forward pass runs
7	Model	gpt.py: embeddings, stacked blocks, final norm, linear head, weight tying	model(x) → (B, T, V) logits
8	Model	Count parameters, print model summary, verify output shapes for both small and medium configs	Param counts match targets
9	Dataset	dataset.py: load .npz, build (x,y) pairs, DataLoader — mandatory batch shift test	Shift test passes ✓
10	Training	train.py skeleton: optimizer, GradScaler, scheduler, wandb init, checkpoint logic	Script runs without error
11	Training	Full training loop: AMP autocast, gradient accumulation, torch.compile(), grad clipping	50-step overfit test passes ✓
12	Training	Launch small model (~30M) full training run. Monitor first 500 steps live in wandb.	Loss curve descending
13	Training	Small model training continues (background). Debug any instabilities. Review loss curve shape.	Run completing stably
14	Scaling	Small model training completes. Save best checkpoint. Start medium model (~60M) training run.	Small model checkpoint saved
15	Scaling	Medium model training (background). Begin evaluation setup in parallel.	Medium run in progress
16	Scaling	Medium model training continues. Write metrics.py (perplexity). Start vocab size experiment (optional).	metrics.py tested on small model
17	Scaling	Medium model training completes. Run perplexity	Perplexity table

		evaluation on both models. Build comparison table.	complete
18	Scaling	Buffer day — handle any training failures, rerun if needed, finalize comparison table.	Results confirmed, backed up
19	Eval	generate.py: implement greedy and top-k sampling. Generate 5 samples per strategy.	Greedy + top-k working
20	Eval	Add top-p (nucleus) and temperature sampling. Generate and compare all four strategies.	All 4 strategies generating
21	Eval	Save all generation samples to text files. Write qualitative analysis comparing outputs.	Sample files saved to repo
22	LSTM (Opt.)	(If on track) Implement 2-layer LSTM baseline, train for same steps as small transformer.	LSTM perplexity recorded
23	LSTM (Opt.)	(If on track) LSTM evaluation complete. Add LSTM row to comparison table. Otherwise: begin cleanup.	Table updated or cleanup started
24	Cleanup	Remove debug code, add docstrings, write unit tests, clean all imports, verify fresh-clone run	Repo runs clean from scratch
25	Report	Write README (arch diagram, results, instructions). Draft report sections 1–6. Final GitHub push.	GitHub repo published

6. Risk Register & Mitigations

These are the failure modes that have ended similar projects prematurely. Read them once now, and again before each phase begins.

Risk	Severity	Mitigation
Causal mask is wrong	CRITICAL	Write the unit test on Day 5 before any training. Print the attention weight matrix for a 6-token sequence and verify lower-triangular structure by eye.
Batch shift (x, y) is off by one	CRITICAL	Print first 5 tokens of x and y from DataLoader. Confirm $y[i] == x[i+1]$ for all i. Do this on Day 9.
LR too high / no scheduler	HIGH	Start with $3e-4$ for small model. If loss explodes or oscillates, halve it. The warmup prevents most LR-related instability.
Training run crashes midway	HIGH	Checkpoint every 500 steps (not every epoch). Training can resume from last checkpoint. Test your checkpoint save/load code before the full run.
VRAM OOM on medium model	MEDIUM	Increase <code>accumulation_steps</code> and reduce per-step batch size. Mixed precision (already in the plan) halves activation memory. Reduce <code>block_size</code> from 256 to 128 as last resort.
Scaling runs take too long	MEDIUM	Start the first training run by Day 12. Run overnight. Do not wait until Phase 5 to start training. Use wandb to monitor from another device.
RoPE implementation is wrong	MEDIUM	Implement RoPE after the base model works and produces reasonable perplexity. Then add RoPE as a modification and verify that perplexity does not get worse (it should improve slightly).
Running out of time at the end	MEDIUM	Days 22–23 are explicitly optional. The LSTM baseline is a bonus, not a requirement. Protect Days 24–25 for cleanup and report no matter what. A clean repo with a good report is worth more than more experiments.

7. Final Deliverables Checklist

Every item on this list should be complete before submitting or presenting. Check them off in order.

Code Repository

- Clean modular codebase matching the folder structure in Section 3
- Single-command training: `python training/train.py --config small`
- Two unit tests (mask test + dataset shift test) in `tests/`
- `requirements.txt` with pinned versions
- Comprehensive README with architecture explanation, results, and generation samples

Training Artifacts

- wandb dashboard with training and validation loss curves for both model sizes
- Best checkpoints for small and medium models saved in `experiments/checkpoints/`
- Perplexity comparison table (small model, medium model, optionally LSTM)

Evaluation Results

- Validation perplexity for both model sizes
- 20+ text generation samples across all four sampling strategies
- Vocabulary size experiment results (8192 vs 16384) if completed

Report

- Introduction covering motivation and scope
- Architecture section with attention math and RoPE explanation
- Methodology: dataset, tokenization choices, training procedure
- Results: scaling study table, perplexity numbers, training curves
- Sampling section: strategy comparison with example outputs
- Discussion: limitations, what scaling laws predict, future work

Final Note

A project that does fewer things cleanly is always stronger than one that does more things sloppily. If you run out of time, protect the core: working model, clean training loop, perplexity results, and a clear report. Everything else is additive.